

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#02)

Implementation of Combinational Circuits using Verilog Gate & Dataflow Level Modeling

Student Name: Hanzala Saeed Roll No: 24i-6535 Section: CE-A

Date performed: 8/26/2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

I created Half Adder and Full Adder using Gate level and Dataflow modeling in Verilog and wrote testbenches to check all input combinations. I simulated the design in ModelSim and I have created the circuit in Logisim also.

Code:

```
//Half adder using gate level modeling

module halfAdder(sum,carry,n1,n2);

input n1;

input n2;

output carry;

output sum;

xor su(sum,n1,n2);

and ca(carry,n1,n2);

endmodule

module testhalfadder();

reg x;

reg y;

wire Sum;

wire Carry;

halfAdder ha(Sum,Carry,x,y);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;
```

```
#50 x=1; y=1;

#50;

end

endmodule

//Full Adder using Gate level modeling

module fullAdder(SUM,Cout,A,B,Cin);

input A;

input B;

input Cin;

output SUM;

output Cout;

wire sum1;

wire c1;

wire c2;

halfAdder ha1(sum1,c1,A,B);

halfAdder ha2(SUM,c2,sum1,Cin);

or ca(Cout,c1,c2);

endmodule

module testFullAdder();

reg x;

reg y;

reg z;

wire Sum;

wire Carry;

fullAdder fa(Sum,Carry,x,y,z);

initial

begin
```

```
x=0; y=0; z=0;

#50 x=0; y=0; z=1;

#50 x=0; y=1; z=0;

#50 x=0; y=1; z=1;

#50 x=1; y=0; z=0;

#50 x=1; y=0; z=1;

#50 x=1; y=1; z=0;

#50 x=1; y=1; z=1;

#50;

end

endmodule

//Half adder using data flow modeling

module halfAdder(sum,carry,n1,n2);

input n1;

input n2;

output carry;

output sum;

assign sum=n1^n2;

assign carry=n1&n2;

endmodule

module testhalfadder();

reg x;

reg y;

wire Sum;

wire Carry;

halfAdder ha(Sum,Carry,x,y);

initial
```

```
begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Full Adder using data flow modeling

module fullAdder(SUM,Cout,A,B,Cin);

input A;

input B;

input Cin;

output SUM;

output Cout;

assign SUM=A^B^Cin;

assign Cout=(A&B)|(B&Cin)|(A&Cin);

endmodule

module testFullAdder();

reg x;

reg y;

reg z;

wire Sum;

wire Carry;

fullAdder fa(Sum,Carry,x,y,z);

initial
```

```
begin

x=0; y=0; z=0;

#50 x=0; y=0; z=1;

#50 x=0; y=1; z=0;

#50 x=0; y=1; z=1;

#50 x=1; y=0; z=0;

#50 x=1; y=0; z=1;

#50 x=1; y=1; z=0;

#50 x=1; y=1; z=1;

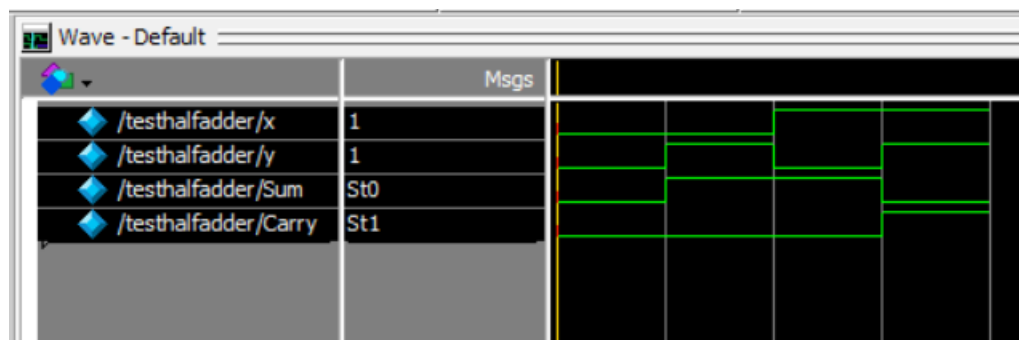
#50;

end

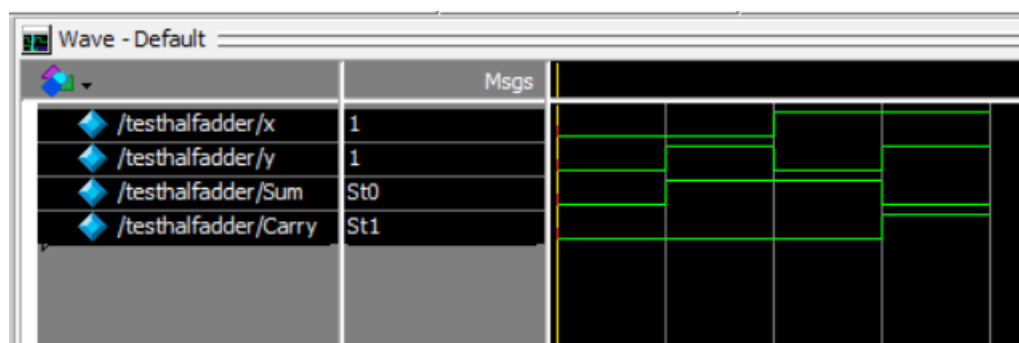
endmodule
```

Output:

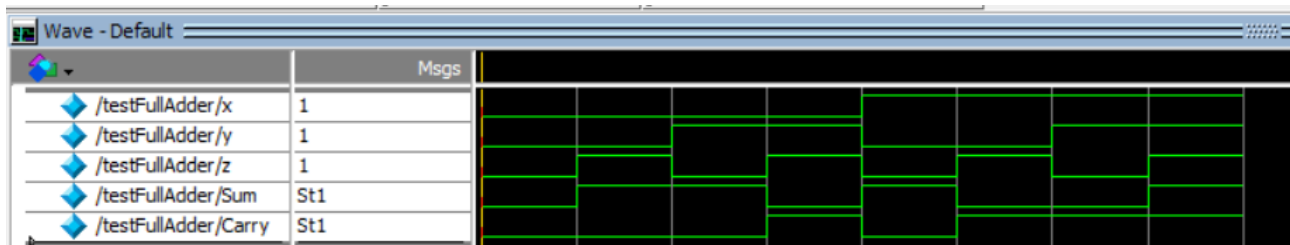
Half-Adder(Gate Level Modeling)



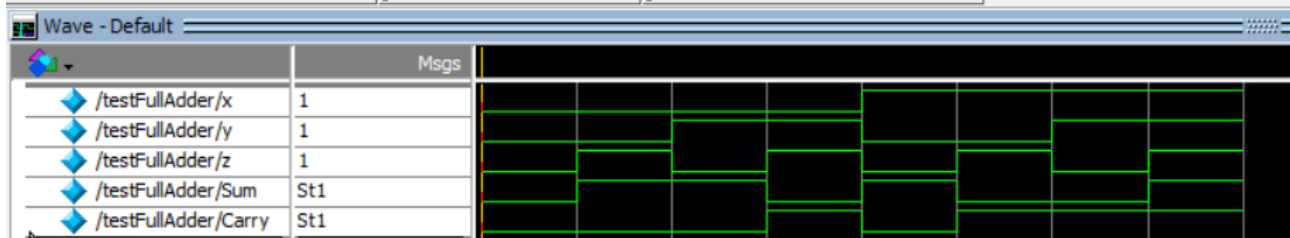
Half-Adder(Date-Flow Modeling)



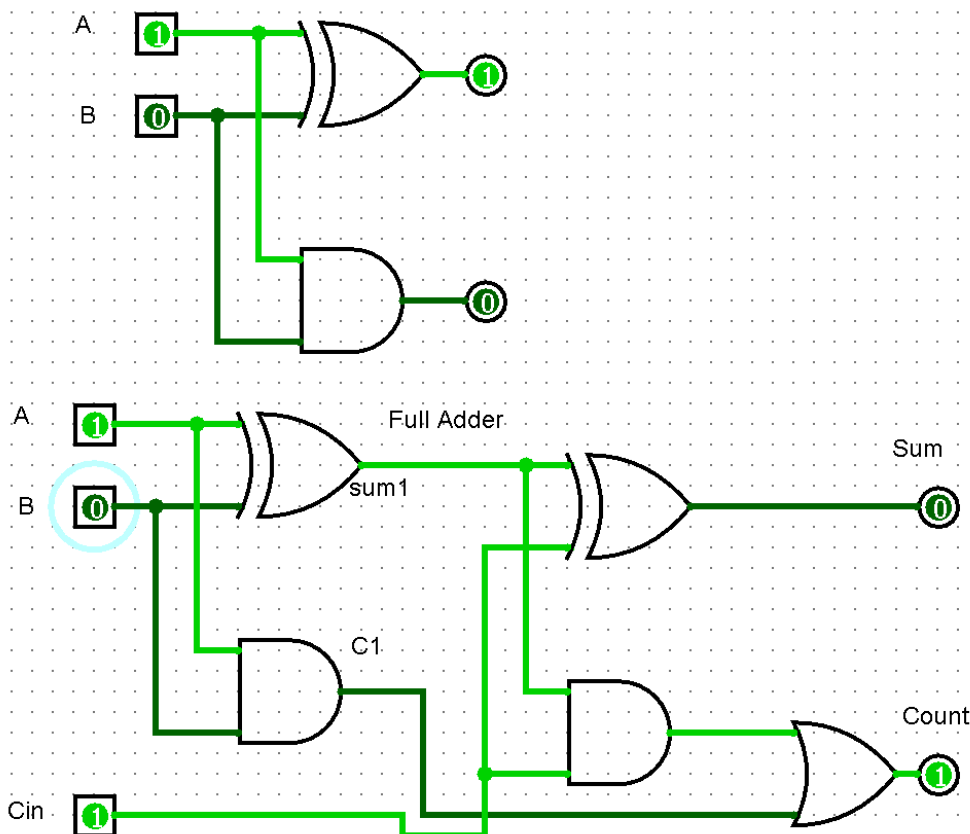
Full-Adder using Gate level modeling



Full-Adder using Data-Flow modeling



Half Adder



Task 2:

Description:

In this task, I created 3×8 Decoder using Gate level and Dataflow modeling in Verilog and wrote testbench to check the result. I simulated it in ModelSim and I have created the circuit of the decoder in Logisim. Output is checked for all combinations of input values.

Code:

```
module a3to8Decoder(o1,o2,o3,o4,o5,o6,o7,o8,A,B,C);

input A,B,C;

output o1,o2,o3,o4,o5,o6,o7,o8;

wire a,b,c;

not (a,A);

not (b,B);

not (c,C);

and (o1,a,b,c);

and (o2,a,b,C);

and (o3,a,B,c);

and (o4,a,B,C);

and (o5,A,b,c);

and (o6,A,b,C);

and (o7,A,B,c);

and (o8,A,B,C);

endmodule

module testDecoder();

reg x;

reg y;

reg z;

wire w1,w2,w3,w4,w5,w6,w7,w8;
```

```
a3to8Decoder a1(w1,w2,w3,w4,w5,w6,w7,w8,x,y,z);
```

```
initial
```

```
begin
```

```
x=0; y=0; z=0;
```

```
#50 x=0; y=0; z=1;
```

```
#50 x=0; y=1; z=0;
```

```
#50 x=0; y=1; z=1;
```

```
#50 x=1; y=0; z=0;
```

```
#50 x=1; y=0; z=1;
```

```
#50 x=1; y=1; z=0;
```

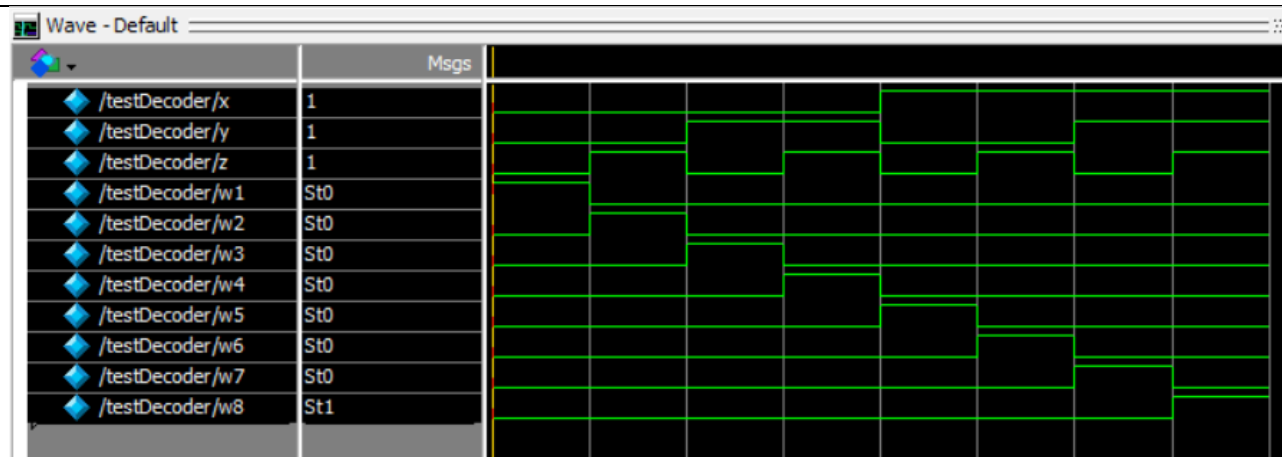
```
#50 x=1; y=1; z=1;
```

```
#50;
```

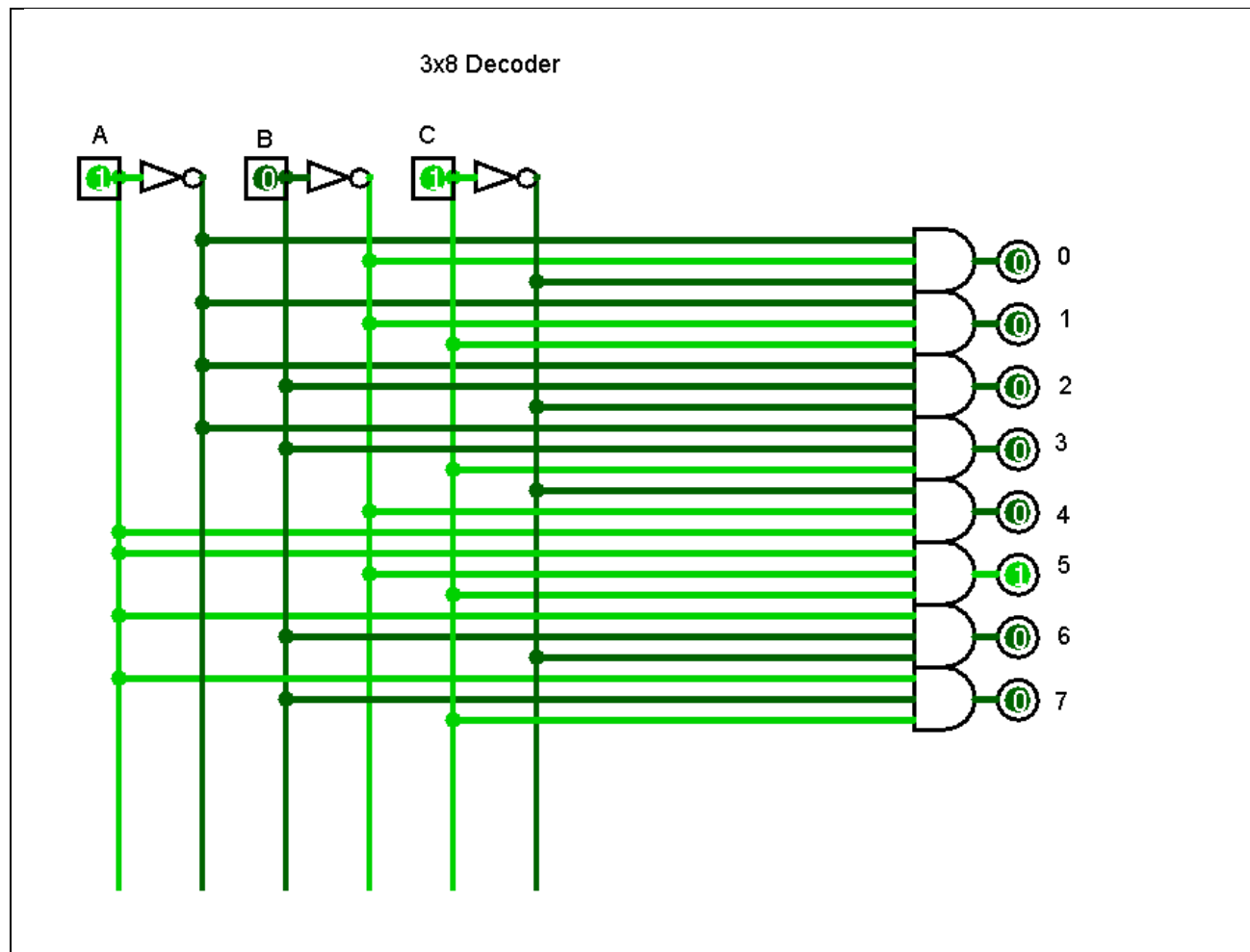
```
end
```

```
endmodule
```

Output:



Circuit Output:



Task 3:

Description:

In this task, I created 8x1 Multiplexer using Gate level and Dataflow modeling in Verilog and wrote testbench to check all combinations of select lines. I simulated the result in ModelSim and I have created the circuit in Logisim also.

Code:

```
//8x1 MUX using Gate level-modeling

module mux8x1Gate(y,d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2);

input d0,d1,d2,d3,d4,d5,d6,d7;

input s0,s1,s2;

output y;

wire n0,n1,n2;

wire a0,a1,a2,a3,a4,a5,a6,a7;
```

```
not g1(n0,s0);

not g2(n1,s1);

not g3(n2,s2);

and g4(a0,d0,n2,n1,n0);

and g5(a1,d1,n2,n1,s0);

and g6(a2,d2,n2,s1,n0);

and g7(a3,d3,n2,s1,s0);

and g8(a4,d4,s2,n1,n0);

and g9(a5,d5,s2,n1,s0);

and g10(a6,d6,s2,s1,n0);

and g11(a7,d7,s2,s1,s0);

or g12(y,a0,a1,a2,a3,a4,a5,a6,a7);

endmodule

module GateLevelTestMux8x1();

reg d0,d1,d2,d3,d4,d5,d6,d7;

reg s0,s1,s2;

wire y;

mux8x1 testgate(y,d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2);

initial

begin

d0=0; d1=1; d2=0; d3=1; d4=0; d5=1; d6=0; d7=1;

s2=0; s1=0; s0=0;

#50 s2=0; s1=0; s0=1;

#50 s2=0; s1=1; s0=0;

#50 s2=0; s1=1; s0=1;

#50 s2=1; s1=0; s0=0;

#50 s2=1; s1=0; s0=1;
```

```
#50 s2=1; s1=1; s0=0;

#50 s2=1; s1=1; s0=1;

#50;

end

endmodule

//8x1 MUX using Data-flow modeling

module mux8x1Data(y,d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2);

input d0,d1,d2,d3,d4,d5,d6,d7;

input s0,s1,s2;

output y;

assign y=(d0&~s2&~s1&~s0)|(d1&~s2&~s1&s0)|(d2&~s2&s1&~s0)|(d3&~s2&s1&s0)|

        (d4&s2&~s1&~s0)|(d5&s2&~s1&s0)|(d6&s2&s1&~s0)|(d7&s2&s1&s0);

endmodule

module DataFlowTestMux8x1();

reg d0,d1,d2,d3,d4,d5,d6,d7;

reg s0,s1,s2;

wire y;

mux8x1 uut(y,d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2);

initial

begin

d0=0; d1=1; d2=0; d3=1; d4=0; d5=1; d6=0; d7=1;

s2=0; s1=0; s0=0;

#50 s2=0; s1=0; s0=1;
```

```
#50 s2=0; s1=1; s0=0;

#50 s2=0; s1=1; s0=1;

#50 s2=1; s1=0; s0=0;

#50 s2=1; s1=0; s0=1;

#50 s2=1; s1=1; s0=0;

#50 s2=1; s1=1; s0=1;

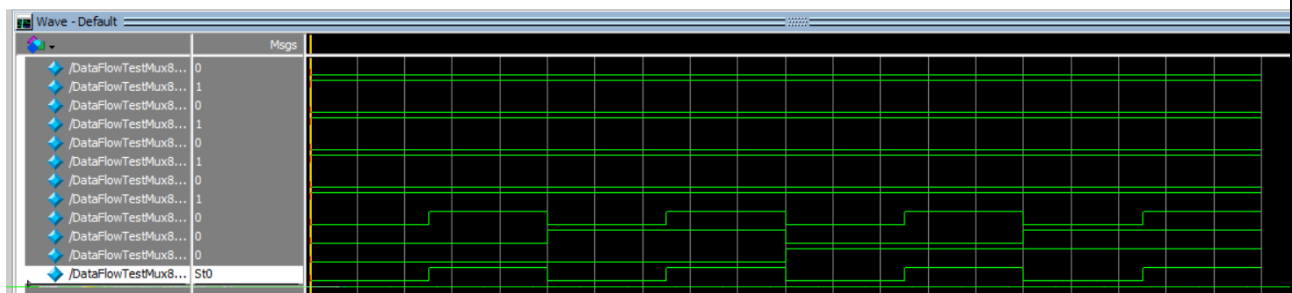
#50;

end

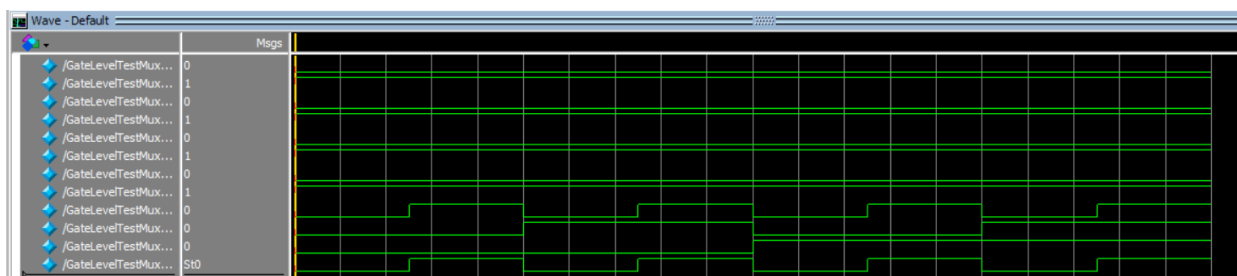
endmodule
```

Output:

Data-Flow Modeling



Gate-Level Modeling



Logisim Circuit:

